5.A. Checking the contents of two files are equal

Problem statement:

Tim is working on a project where he needs to compare the contents of two binary files named 'file1.bin' and 'file2.bin'.

Your task is to help Tim write a program that takes the binary content for both files as input from the user, compares their contents, and checks if the contents of the two files are equal.

Input format :

The first line of input consists of the content for file1.bin as a binary string.

The second line consists of the content for file2.bin as a binary string.

Output format :

If the contents of file1.bin and file2.bin are equal, print "Two files are equal". Otherwise, print "Two files are not equal".

Refer to the sample output for formatting specifications.

Code constraints :

Length of the Binary String = 8

Aim:

To read two binary strings from user input, write their corresponding byte values to separate binary files (file1.bin and file2.bin), and then compare the contents of these files to determine if they are equal.

Algorithm:

1. Start
2. Read two binary strings from user input.
3. Convert each binary string to its decimal value.
4. Write each decimal value to separate binary files (file1.bin and file2.bin).
5. Read one byte from each file.
6. Compare the bytes:
7. If equal → print "Two files are equal"
8. Else → print "Two files are not equal"
9. End

Program:

```
    import java.util.Scanner;

public class Main {
  public static void main(String[] args) {
    Scanner sc = new Scanner(System.in);

    String file1 = sc.nextLine();
    String file2 = sc.nextLine();

    if (file1.equals(file2)) {
      System.out.println("Two files are equal");
    } else {
      System.out.println("Two files are not equal");
    }
  }
}
```

Fill your code here Java (11)

```java
import java.util.Scanner;

public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        String file1 = sc.nextLine();
        String file2 = sc.nextLine();

        if (file1.equals(file2)) {
            System.out.println("Two files are equal");
        } else {
            System.out.println("Two files are not equal");
        }
    }
}
```

Output:

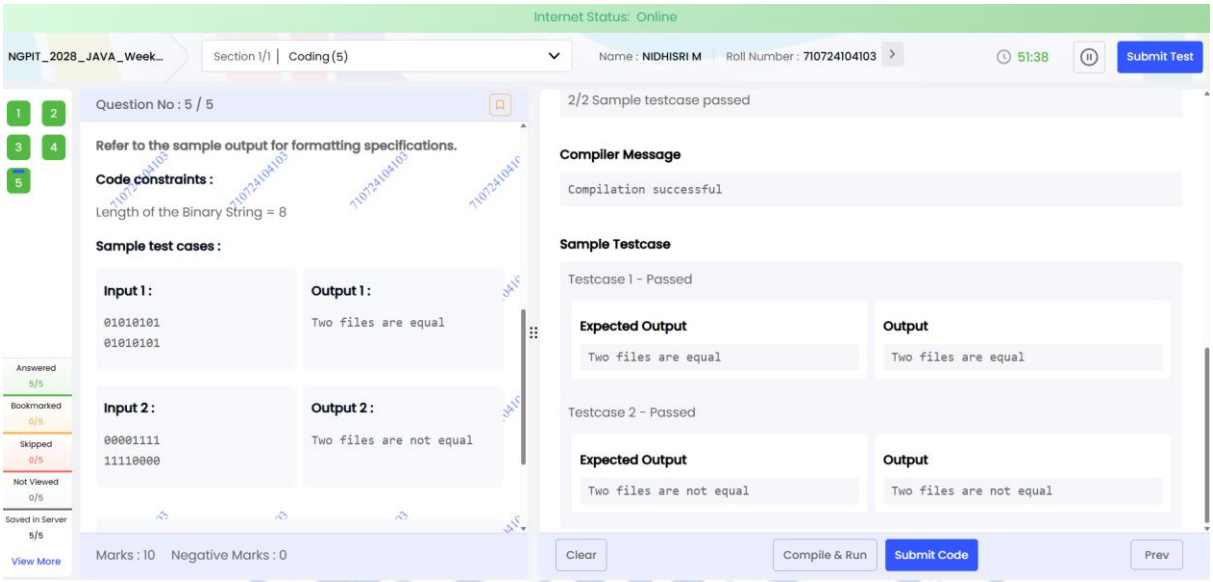
Input 1 :

01010101

01010101

Output 1 :

Two files are equal



Result:

Thus the above java program to read two binary strings from user input, write their corresponding byte values to separate binary files has been completed successfully.

5.B.Finding Minimum Value from File Data

Problem statement:

Arun wants to create a program that takes input from the user, writes the entered values into a file named "output.txt", and then reads the values from the file to find and print the minimum value.

Help Arun to complete the program.

Input format :

The first line of input consists of an integer N, representing the number of elements in the array.
The second line consists of N space-separated array elements.

Output format :

The output prints an integer which is the minimum element in the array.

Refer to the sample output for formatting specifications.
Code constraints :

$1 \leq N \leq 10$
$1 \leq$ Array Elements ≤ 100

Aim:

To write a program that accepts N elements from the user, stores them in a file named "output.txt", then reads the values from the file to find and display the minimum element in the array.

Algorithm:

1. Start the program
2. Input N array elements from the user.
3. Open a file named "output.txt" in write mode.
4. Reopen the file in read mode.
5. Convert the values into integers.
6. Find the minimum value from the list.
7. Stop the program

Program:

```
import java.io.*;
import java.util.*;

public class Main {
   public static void main(String[] args) throws IOException {
      Scanner sc = new Scanner(System.in);
      int n = sc.nextInt();
      FileWriter fw = new FileWriter("output.txt");
      for (int i = 0; i < n; i++) fw.write(sc.nextInt() + " ");
      fw.close();

      Scanner file = new Scanner(new File("output.txt"));
      int min = 101;
      while (file.hasNextInt()) min = Math.min(min, file.nextInt());
      file.close();
      System.out.print(min);
   }
}
```

Fill your code here Java (11)

```java
import java.io.*;
import java.util.*;

public class Main {
    public static void main(String[] args) throws IOException {
        Scanner sc = new Scanner(System.in);
        int n = sc.nextInt();
        FileWriter fw = new FileWriter("output.txt");
        for (int i = 0; i < n; i++) fw.write(sc.nextInt() + " ");
        fw.close();

        Scanner file = new Scanner(new File("output.txt"));
        int min = 101;
        while (file.hasNextInt()) min = Math.min(min, file.nextInt());
        file.close();
        System.out.print(min);
    }
}
```

Output:

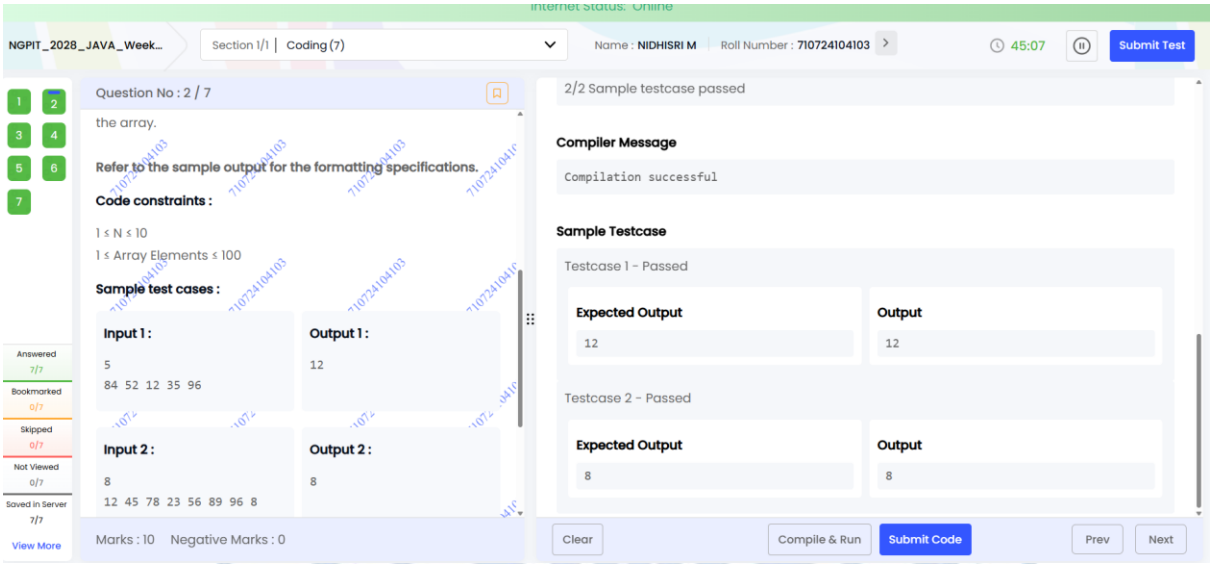
Input 1 :

5

84 52 12 35 96

Output 1 :

12



Result:

Thus, the program is successfully executed.The given N numbers are written into the file "output.txt" and then read back to find and display the minimum element in the array.

5.C.Finding Words with Vowels from File Data

Problem statement:

Yuvana is fascinated by text analysis and wants to create a program that identifies words in a given text containing at least one vowel. She envisions a program that takes a string as input, saves it to a file named "output.txt", and then analyzes the content to display words containing vowels.

Your task is to help Yuvana in developing this program.

Input format :

The input consists of a string.

Output format :

The output prints the words from the input string that contains at least one vowel.
Each identified word is printed on a new line.

Refer to the sample output for formatting specifications.

Code constraints :

1 ≤ Length of the input string ≤ 100

Aim:

To write a program that takes a string as input, stores it in a file named "output.txt", then reads the content from the file and displays all the words that contain at least one vowel.

Algorithm:

1. Start the program.
2. Input a string from the user.
3. Open a file named "output.txt" in write mode.
4. Write the input string into the file.
5. Close the file.
6. Reopen the file in read mode.
7. Read the content from the file.
8. Stop the program,

Program:

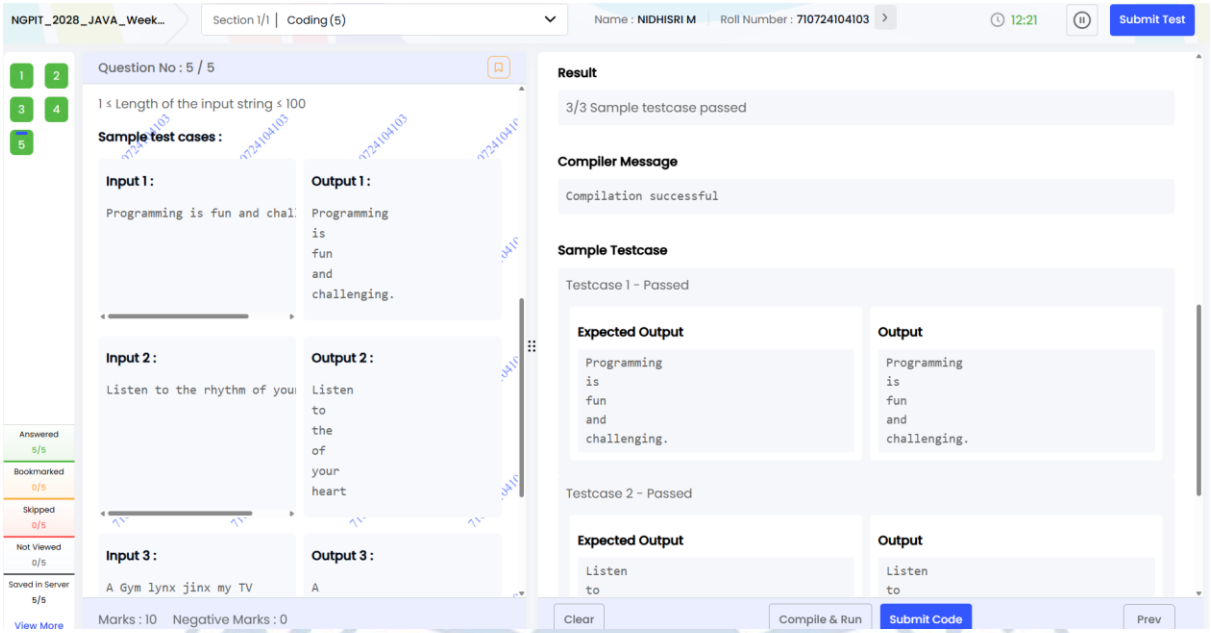
```
import java.io.*;

public class Main {
   public static void main(String[] args) throws IOException {
      BufferedReader br = new BufferedReader(new InputStreamReader(System.in));
      String s = br.readLine();

      FileWriter fw = new FileWriter("output.txt");
      fw.write(s);
      fw.close();

      FileReader fr = new FileReader("output.txt");
      StringBuilder sb = new StringBuilder();
      int ch;
```

```
        while ((ch = fr.read()) != -1) sb.append((char) ch);
        fr.close();

        for (String word : sb.toString().split(" "))
            if (word.matches(".*[aeiouAEIOU].*"))
                System.out.println(word);
    }
}
```

```java
import java.io.*;

public class Main {
    public static void main(String[] args) throws IOException {
        BufferedReader br = new BufferedReader(new InputStreamReader(System.in));
        String s = br.readLine();

        FileWriter fw = new FileWriter("output.txt");
        fw.write(s);
        fw.close();

        FileReader fr = new FileReader("output.txt");
        StringBuilder sb = new StringBuilder();
        int ch;
        while ((ch = fr.read()) != -1) sb.append((char) ch);
        fr.close();

        for (String word : sb.toString().split(" "))
            if (word.matches(".*[aeiouAEIOU].*"))
                System.out.println(word);
    }
}
```

Output:

Input 1 :
Programming is fun and challenging.
Output 1 :
Programming
is
fun
and
challenging.



Result:

Thus, the program was successfully executed.The given input string was written into the file "output.txt" and the words containing at least one vowel were identified and displayed.